

Fórmulas

Formulas estadísticas

Estas funciones estadísticas son los "ladrillos" matemáticos con los que se construyen muchos procesos del Aprendizaje Automático, desde la normalización de datos hasta las decisiones internas de los algoritmos.

Aquí tienes las fórmulas y su aplicación exacta dentro de este contexto:

1. Media Aritmética (μ o $\bar{y}$)

Es el valor promedio matemático de un conjunto de datos.

- **Fórmula:** $\bar{y} = \frac{1}{|D|} \sum_{(x,y) \in D} y$. (Es decir, se suman todos los valores y se divide el resultado entre la cantidad total de ejemplos $|D|$).
- **¿Para qué se usa en Machine Learning?**
 - **Normalización (Tipificación):** Es el valor central (μ) que se le resta a cada atributo numérico para centrar los datos.
 - **Predicciones de Regresión:** En algoritmos como kNN, la predicción final es la media de los valores de los vecinos cercanos. De igual forma, en los árboles CART de regresión, cada nodo hoja se etiqueta usando la media de los valores que han caído en él.

2. Varianza ($Var(D)$ o σ^2)

Mide cómo de "dispersos" o separados están los datos con respecto a su propia media. Matemáticamente, es el promedio de las diferencias al cuadrado entre cada valor individual y la media general.

- **Fórmula:** $Var(D) = \frac{1}{|D|} \sum_{(x,y) \in D} (y - \bar{y})^2$.
- **¿Para qué se usa en Machine Learning?**
 - **Árboles CART de Regresión:** La varianza se utiliza como la **función de impureza**. El árbol busca dividir los datos intentando que los subconjuntos resultantes tengan la varianza más pequeña posible. Si en una rama todos los ejemplos tienen exactamente el mismo valor numérico, la varianza es cero y se considera un nodo "puro".
 - **Coefficiente de Determinación (R^2):** Se utiliza en el denominador de su fórmula para comparar el error de la red neuronal con la varianza natural de los datos.

3. Desviación Típica (σ)

Es la medida directa de dispersión que acompaña a la media.

- **Fórmula / Concepto:** Aunque a nivel conceptual es la raíz cuadrada de la varianza, en los apuntes se denota universalmente como σ .
- **¿Para qué se usa en Machine Learning?**

- Es el divisor clave en la **Tipificación** (o Z-score): $x_{norm} = \frac{x - \mu}{\sigma}$. Se utiliza para escalar o comprimir los atributos numéricos de forma que todos terminen teniendo exactamente desviación típica 1 y media 0.

Un apunte práctico para el ordenador: Aunque dominar la teoría es vital para los ejercicios en papel, cuando resuelvas problemas programando en Python con la biblioteca **Pandas**, todo esto está automatizado.

- Puedes calcular la media aritmética invocando directamente la función `.mean()` sobre tus datos.
- Si aplicas la función `.describe()` a un `DataFrame`, Python te devolverá al instante una tabla estadística completa que incluye la media (`mean`), la desviación típica (`std`), y los valores mínimos y máximos de todas tus columnas de datos.

Tienes toda la razón, mis disculpas. Al centrarme en explicarte los conceptos y para qué servían, omití darte sus ecuaciones matemáticas exactas. Aquí las tienes detalladas paso a paso:

1. Coeficiente de Determinación (R^2)

Esta métrica evalúa la calidad del ajuste de un modelo de regresión comparando el error que comete tu modelo con la varianza natural que ya tienen los datos reales.

- **Fórmula general:**

$$R^2 = 1 - \frac{MSE}{Var(D)}$$

- **Fórmula extendida:** Sabiendo que el MSE es el Error Cuadrático Medio y $Var(D)$ es la varianza, si desarrollamos ambos cálculos obtenemos la ecuación completa:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

(Nota matemática: La constante $\frac{1}{|D|}$ que va delante de las sumatorias del MSE y la varianza se cancela al dividir, por lo que la fórmula se simplifica a la suma de errores al cuadrado entre la suma de varianzas).

- **Significado de las variables:** y es el valor real esperado, $\hat{y}$ es el valor que ha predicho tu modelo y $\bar{y}$ es la media aritmética de todos los valores reales.

2. Desviación Típica o Estándar (σ)

Como te adelantaba en el mensaje anterior, la desviación típica se define matemáticamente como la raíz cuadrada positiva de la varianza.

- **Fórmula general:**

$$\sigma = \sqrt{Var(D)}$$

- **Fórmula extendida:**

$$\sigma = \sqrt{\frac{1}{|D|} \sum (y - \bar{y})^2}$$

- **Significado de las variables:** $|D|$ es la cantidad total de ejemplos (o datos), y es cada valor numérico individual y $\bar{y}$ es la media aritmética de esos datos.

A nivel práctico para resolver problemas a mano, mi consejo es que calcules primero la varianza sumando todas las diferencias al cuadrado y dividiendo entre el total de datos y, una vez tengas ese número único, le apliques la raíz cuadrada en tu calculadora para obtener σ .

Aprendizaje automático: Naives Bayes

El modelo Naive Bayes se fundamenta en la **regla de decisión de máximo a posteriori (MAP)**, realizando la asunción matemática de que los atributos son independientes entre sí dentro de cada clase.

Aquí tienes todas las fórmulas clave que necesitas para entrenar y aplicar este modelo:

1. La Fórmula Maestra de Predicción (Regla MAP)

Para predecir a qué clase $\hat{c}$ pertenece un nuevo ejemplo compuesto por varios atributos ($x_1, x_2, \dots, x_n$), se utiliza la siguiente ecuación:

$$\hat{c} = \arg \max_{c \in C} \mathbb{P}(c) \prod_{i=1}^n \mathbb{P}(X_i = x_i | c)$$

¿Qué significa de forma intuitiva? Calculas la puntuación para cada clase posible multiplicando su probabilidad de partida $\mathbb{P}(c)$ por la probabilidad condicional de cada uno de los atributos del ejemplo nuevo $\mathbb{P}(X_i = x_i | c)$. Finalmente, te quedas con la clase que obtenga el valor numérico más alto (eso es lo que hace la orden **arg max**).

2. Fórmulas de Entrenamiento (Estimación de Parámetros)

Para poder usar la fórmula anterior, el modelo debe aprender primero las probabilidades desde tu conjunto de datos de entrenamiento.

- **Probabilidad a priori de la clase:**

$$\mathbb{P}(c) = \frac{N_c}{N}$$

(Se calcula dividiendo el número de ejemplos de esa clase, N_c , entre el total de ejemplos del conjunto, N).

- **Probabilidad condicional estándar (sin suavizado):**

$$\mathbb{P}(X = x|c) = \frac{N_{X=x,c}}{N_c}$$

(Se calcula dividiendo cuántas veces aparece el valor x dentro de la clase c , entre el número total de ejemplos de esa clase).

3. Fórmula del Suavizado de Laplace (Imprescindible)

Si un atributo tiene un valor que nunca apareció en el entrenamiento para una clase concreta, la fórmula estándar daría una probabilidad de 0, lo que anularía por completo toda la multiplicación en la regla MAP. Para solucionar esto, **siempre se recomienda usar el suavizado de Laplace**:

$$\mathbb{P}(X = x|c) = \frac{N_{X=x,c} + 1}{N_c + |X|}$$

- $+1$: Se suma un ejemplo "virtual" al numerador para evitar el cero.
- $|X|$: Es la cantidad total de valores distintos que puede llegar a tomar el atributo X , y se suma al denominador para mantener la proporción matemática correcta. (Nota: Existe una versión generalizada llamada "suavizado aditivo" donde en lugar de 1, sumas una constante k arriba y $k|X|$ abajo).

4. Transformación Logarítmica (Para evitar fallos del ordenador)

Cuando tienes una gran cantidad de atributos, multiplicar muchas probabilidades (que son números pequeños entre 0 y 1) genera un número tan minúsculo que el ordenador no puede procesarlo y lo redondea a cero (esto se llama *underflow numérico*).

Para evitarlo en la práctica, la regla MAP se transforma matemáticamente aplicando logaritmos, lo que **convierte las multiplicaciones en sumas** sin alterar cuál será la clase ganadora:

$$\hat{c} = \arg \max \left(\log \mathbb{P}(c) + \sum_{i=1}^n \log \mathbb{P}(X_i = x_i|c) \right)$$

5. Adaptación para Procesamiento de Lenguaje Natural (Textos)

Si estás usando Naive Bayes para clasificar textos (como correos spam o análisis de sentimientos), la fórmula del suavizado de Laplace se adapta ligeramente al modelo de "Bolsa de palabras". En lugar de contar ejemplos enteros, cuentas repeticiones de palabras:

$$\mathbb{P}(t|c) = \frac{\text{Ocurrencias de la palabra } t \text{ en la clase } c + 1}{\text{Total de palabras en la clase } c + |V|}$$

(Donde $|V|$ es el tamaño del vocabulario, es decir, el número de palabras únicas en todos los textos). Y en la regla MAP, elevarás esta probabilidad al número de veces que dicha palabra aparezca repetida en el documento nuevo que quieres clasificar.

El algoritmo de los k vecinos más cercanos (kNN) es mucho más geométrico e intuitivo que Naive Bayes. Puesto que es un modelo no paramétrico que simplemente memoriza los datos de entrenamiento para compararlos, sus fórmulas no buscan probabilidades, sino que se centran puramente en **calcular distancias** y en **normalizar escalas**.

Aquí tienes las fórmulas matemáticas que rigen el algoritmo:

1. Fórmulas de Similitud (Métricas de Distancia)

Para decidir qué ejemplos son los "más cercanos", kNN necesita medir el espacio entre el ejemplo nuevo (x) y un ejemplo de entrenamiento (x'). Dependiendo del tipo de datos, se usa una de estas tres fórmulas:

- **Distancia Euclídea (para atributos numéricos):** Calcula la distancia geométrica clásica en "línea recta".

$$d(x, x') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2}$$

- **Distancia Manhattan (para atributos numéricos):** Calcula la distancia sumando las diferencias absolutas a lo largo de cada eje (como si te movieras por las manzanas de una ciudad).

$$d(x, x') = \sum_{i=1}^n |x_i - x'_i|$$

- **Distancia de Hamming (para atributos discretos):** Se usa cuando los atributos no son números (por ejemplo: colores, tamaños o categorías). Simplemente cuenta en cuántos atributos difieren ambos ejemplos.

$$d(x, x') = \sum_{i=1}^n \mathbb{I}(x_i \neq x'_i)$$

(La función $\mathbb{I}$ vale 1 si los atributos son distintos, y 0 si son exactamente iguales).

2. Fórmulas de Normalización (Imprescindibles)

Como ya vimos cuando resolvimos juntos el Ejercicio 16, si no aplicas una normalización previa, un atributo numérico con un rango muy grande (como el peso de un coche o el precio de un viaje) dominará por completo el cálculo de la distancia sobre los atributos más pequeños. Las dos fórmulas típicas para solucionar esto son:

- **Tipificación:** (Esta es la que usamos en tu Ejercicio 16). Convierte los datos para que tengan media 0 y desviación típica 1.

$$x_{norm} = \frac{x - \mu}{\sigma}$$

- **Normalización Mín-Máx:** Comprime todos los valores estrictamente dentro del intervalo entre 0 y 1.

$$x_{norm} = \frac{x - m}{M - m}$$

(Donde m es el valor mínimo del atributo y M es el máximo).

3. Fórmulas de Predicción (La regla de decisión final)

Una vez que el algoritmo ha calculado las distancias y ha aislado a los k ejemplos de entrenamiento más cercanos (por ejemplo, los 3 o 5 más próximos), toma la decisión final dependiendo del tipo de problema:

- **Para problemas de Clasificación:** El modelo asigna la **clase mayoritaria** (la moda). Simplemente se hace una votación entre los k vecinos y gana la clase que más se repita. *(Nota teórica: Por eso siempre se aconseja que el parámetro k sea un número impar en clasificación binaria, ¡para evitar empates en la votación!).*
- **Para problemas de Regresión:** Si intentas predecir un valor continuo, el modelo calcula la **media aritmética** de los valores asociados a los k vecinos cercanos.

$$\hat{y} = \frac{1}{k} \sum_{j=1}^k y_j$$

Árboles de Decisión (CART)

El algoritmo CART (Árboles de Clasificación y Regresión) basa todo su aprendizaje en buscar matemáticamente el "corte" perfecto para ir dividiendo el conjunto de entrenamiento. Para lograrlo, utiliza funciones que miden la **impureza** de los datos.

Aquí tienes las fórmulas fundamentales que rigen el algoritmo:

1. La Fórmula Maestra de Partición (Impureza Promedio)

Cuando el árbol está en un nodo e intenta averiguar por dónde cortar un atributo usando un valor umbral (u), divide temporalmente sus datos en dos ramas (izquierda y derecha). El objetivo del algoritmo es encontrar el corte que minimice la siguiente fórmula de impureza ponderada:

$$\text{Impureza} = \frac{|\mathcal{D}^{Izq}|}{|\mathcal{D}|} I(\mathcal{D}^{Izq}) + \frac{|\mathcal{D}^{Der}|}{|\mathcal{D}|} I(\mathcal{D}^{Der})$$

(Donde $|\mathcal{D}|$ es la cantidad total de ejemplos en el nodo que vas a dividir, $|\mathcal{D}^{Izq}|$ y $|\mathcal{D}^{Der}|$ son cuántos ejemplos caen a cada lado del corte, e I es la función de impureza que aplique según el tipo de problema).

2. Fórmulas para tareas de Clasificación

Si tu objetivo es predecir una categoría (ejemplo: "spam" o "no spam"), la función de impureza I que utiliza CART por defecto es el **Índice de Gini**.

- **Fórmula del Índice de Gini:**

$$G(D) = 1 - \sum_{c \in C} \hat{\Pi}_c^2$$

(Donde $\hat{\Pi}_c$ es la proporción de ejemplos que pertenecen a la clase c dentro de ese nodo). Si todos los datos de una partición son de la misma clase, el índice de Gini dará exactamente 0, lo que indica pureza total.

- **Predicción final (Hojas):** Cuando el árbol no se puede dividir más, la etiqueta final del nodo hoja se decide calculando la **clase mayoritaria** de los ejemplos que han caído en él.

3. Fórmulas para tareas de Regresión

Si tu objetivo es predecir un número continuo (ejemplo: el precio de una casa o los mililitros de helado vendidos), la función de impureza I que asume el algoritmo es la **Varianza**.

- **Fórmula de Impureza (Varianza):**

$$Var(\mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} (y - \bar{y})^2$$

(El algoritmo buscará particiones que hagan que esta varianza sea lo más cercana a 0 posible, es decir, agrupando ejemplos con valores numéricos muy similares).

- **Predicción final (Hojas):** La predicción que devolverá la hoja del árbol para cualquier dato nuevo será matemáticamente la **media aritmética** de los valores objetivos de los ejemplos de entrenamiento asociados a esa hoja:

$$ETIQUETA(\mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} y = \bar{y}$$

Redes Neuronales

Funciones de activación

Las principales funciones de activación diferenciables utilizadas en neuronas artificiales (descartando las funciones umbral y signo propias del perceptrón clásico) son las siguientes:

Función de Activación	Expresión Matemática	Derivada matemática
Sigmoide (σ)	$\frac{1}{1 + e^{-z}}$	$\sigma(z)(1 - \sigma(z))$
Tangente Hiperbólica ($\tanh$)	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \tanh^2(z)$
Rectificador (ReLU)	$\max(0, z)$	$0 \text{ si } z \leq 0, 1 \text{ si } z > 0$
Softmax	$\frac{e^{z_k}}{\sum e^{z_i}}$	Matriz Jacobiana compleja (depende de todas las salidas).

Para que una red neuronal pueda entrenarse utilizando el método del descenso por el gradiente y el algoritmo de retropropagación, es un **requisito indispensable que la función de activación elegida sea diferenciable**. El uso de estas funciones permite que pequeños ajustes en los pesos de las neuronas produzcan cambios suaves en la salida, superando así la gran limitación expresiva del perceptrón.

Funciones de coste

Función de Coste	Expresión Matemática	Derivada (respecto a la salida a)
Error Cuadrático Medio (MSE)	$\frac{1}{n} \sum_{k=1}^n (y_k - a_k)^2$	$\frac{2}{n} (a - y)$
Entropía Cruzada Binaria	$-y \log_e(a) - (1 - y) \log_e(1 - a)$	$\frac{a - y}{a(1 - a)}$

Función de Coste	Expresión Matemática	Derivada (respecto a la salida a)
Entropía Cruzada Categórica	$-\sum_{k=1}^n y_k \log_e(a_k)$	$-\frac{y}{a}$

Métricas de error

El **coeficiente de determinación** (universalmente conocido en estadística y Machine Learning como R^2) es una métrica para evaluar cómo de bueno es un modelo de regresión.

Mientras que el Error Cuadrático Medio te da un valor absoluto (que a veces es difícil de interpretar por sí solo), el R^2 te da una "nota" proporcional:

- Un $R^2 = 1.0$ significa que la red neuronal hace predicciones perfectas sin margen de error.
- Un $R^2 = 0.0$ significa que la red neuronal es tan mala que obtendrías los mismos resultados si ignorases la red y siempre predijeras la media aritmética de los datos.
- Incluso puede ser negativo si el modelo es peor que predecir simplemente la media.

(Nota importante: Debo aclararte que la definición y la fórmula matemática exacta de este coeficiente no vienen incluidas en los apuntes teóricos de Redes Neuronales que tengo en mis fuentes, por lo que te estoy dando esta información basándome en conocimiento matemático externo general. Te aconsejo verificar si tu profesor utiliza alguna variante específica de la fórmula en sus transparencias, aunque el estándar es universal).

La fórmula matemática clásica es:

$$R^2 = 1 - \frac{\text{Suma de los errores al cuadrado}}{\text{Suma total de las varianzas}}$$

$$R^2 = 1 - \frac{\sum (y_i - a_i)^2}{\sum (y_i - \bar{y})^2}$$

El coeficiente de determinación es una métrica **global** que solo se puede sacar cuando hayas terminado de pasar por la red los **5 ejemplos** de la tabla del Ejercicio 11.

Ciclo completo de fórmulas para Redes Neuronales para resolver ejercicios de retropropagación

Para resolver un ejercicio completo en tu examen, el gran secreto que debes tener en cuenta es que **gran parte del algoritmo es universal**. Las fórmulas matemáticas para propagar la información por las capas ocultas y para actualizar los pesos son exactamente las mismas en las tres configuraciones; lo único que cambia es cómo actúa y cómo se equivoca la capa de salida.

A continuación tienes el ciclo completo de fórmulas matriciales paso a paso, integrando tus tres configuraciones:

FASE 1: Propagación hacia adelante (Forward)

Para calcular cómo transita la información desde la primera capa oculta hasta la salida, se aplica la misma operación en cada capa l :

1. **Entrada neta:** $z^l = W^l a^{l-1} + W_0^l$
2. **Activación:** $a^l = g^l(z^l)$

Lo que cambia: En las capas ocultas, g^l suele ser la función Rectificador (ReLU) o Sigmoide. Sin embargo, en la capa de salida (L), g^L dependerá de tu configuración:

- **Regresión:** $a^L = z^L$ (Función Identidad).
- **Clasificación Binaria:** $a^L = \frac{1}{1+e^{-z^L}}$ (Función Sigmoide).
- **Clasificación Multiclase:** $a^L = \frac{e^{z_i^L}}{\sum_j e^{z_j^L}}$ (Función Softmax)

FASE 2: Cálculo del Coste y del Error Inicial (Δ^L)

Aquí es donde aplicamos las tres reglas de oro que mencionaste. Comparamos tu predicción (a^L) con el objetivo real (y):

- **1. Regresión**
 - Cálculo del coste: $MSE = \frac{1}{n} \sum_{k=1}^n (y_k - a_k^L)^2$
 - Error a propagar: $\Delta^L = \frac{2}{n} (a^L - y)$
- **2. Clasificación Binaria**
 - Cálculo del coste: $C = -y \log_e(a^L) - (1 - y) \log_e(1 - a^L)$
 - Error a propagar: $\Delta^L = a^L - y$
- **3. Clasificación Multiclase**
 - Cálculo del coste: $C = -\sum_{k=1}^n y_k \log_e(a_k^L)$
 - Error a propagar: $\Delta^L = a^L - y$

FASE 3: Propagación hacia atrás del error (Backpropagation)

Una vez que tienes el vector Δ^L calculado en el paso anterior, esta fórmula matricial vuelve a ser **completamente universal** para cualquiera de los tres problemas. Para trasladar el error a las capas interiores ($l = L - 1, \dots, 2$), aplicas:

$$\Delta^l = ((W^{l+1})^T \Delta^{l+1}) \odot (g^l)'(z^l)$$

(Recuerda: multiplicas el error de la capa superior por los pesos transpuestos, y luego aplicas el producto de Hadamard $\odot$ multiplicando posición a posición por la derivada de la activación de esa capa oculta).

FASE 4: Recálculo de Pesos (Gradientes y Descenso Estocástico)

Teniendo las activaciones de la Fase 1 y los errores de la Fase 3, se calculan los gradientes locales que dictan cuánto debe cambiar la red para este ejemplo concreto. Las fórmulas **universales** son:

- **Gradiente de los pesos:** $\frac{\partial C}{\partial W^l} = \Delta^l (a^{l-1})^T$
- **Gradiente de los sesgos:** $\frac{\partial C}{\partial W_0^l} = \Delta^l$

Finalmente, como los ejercicios suelen pedir ajustar los parámetros por "minilotes" (un bloque de m ejemplos juntos), sumas los gradientes de todos los ejemplos del bloque y aplicas la regla de actualización multiplicada por la tasa de aprendizaje η :

- **Nuevos Pesos:** $W^l \leftarrow W^l - \frac{\eta}{m} \sum \Delta^l (a^{l-1})^T$
- **Nuevos Sesgos:** $W_0^l \leftarrow W_0^l - \frac{\eta}{m} \sum \Delta^l$

PNL - Procesamiento de Lenguaje Natural

CONFIGURACIÓN 1: Clasificación de Documentos (Bolsa de Palabras + Naive Bayes)

Se usa cuando nos importa la frecuencia entera de repetición de cada palabra sin discriminar su rareza general.

1. Entrenamiento del Modelo (Cálculo de parámetros):

- **Probabilidad a priori de la clase:** $\mathbb{P}(c) = \frac{N_c}{N}$ (*Documentos de la clase c entre el total de documentos*).
- **Probabilidad condicional (con Suavizado Laplace):**

$$\mathbb{P}(t|c) = \frac{N_{c,t} + 1}{\sum_{s \in V} N_{c,s} + |V|}$$

(*Ocurrencias de la palabra t en la clase c + 1, dividido entre el total de palabras en la clase c + el tamaño del vocabulario $|V|$*).

2. Predicción de un nuevo documento: Para clasificar un texto nuevo, usas la versión con logaritmos para evitar desbordamientos, multiplicando por el número de veces ($n_{D,t}$) que aparece la palabra en el texto de prueba:

$$\hat{c} = \arg \max_{c \in C} \left(\log \mathbb{P}(c) + \sum_{t \in V} n_{D,t} \log \mathbb{P}(t|c) \right)$$

CONFIGURACIÓN 2: Clasificación de Documentos (TF-IDF + kNN)

Se usa cuando queremos darle más importancia a las palabras "clave" o raras del texto y penalizar las demasiado comunes (como preposiciones o artículos).

1. Representación del Documento (Pesos TF-IDF): Para cada palabra t en el documento D :

$$tf-idf_{t,D} = tf_{t,D} \times \log_2 \left(\frac{N}{df_t} \right)$$

(Donde tf es las veces que aparece t en el documento, N es el total de documentos de entrenamiento, y df es en cuántos documentos distintos aparece la palabra t).

2. Predicción de un nuevo documento:

- **Métrica (Similitud del Coseno):** Se calcula el coseno entre el vector del documento nuevo y cada ejemplo de entrenamiento:

$$sim(D_1, D_2) = \frac{D_1 \cdot D_2}{\|D_1\|_2 \times \|D_2\|_2}$$

- **Regla de Decisión:** Te quedas con los k documentos que saquen el valor de similitud más alto y asignas la clase mayoritaria entre ellos.

CONFIGURACIÓN 3: Modelos de n-gramas (Predicción de secuencias)

Aquí el objetivo es predecir secuencias asumiendo que cada palabra depende solo de las $n - 1$ palabras anteriores (propiedad de Markov).

1. Fórmulas de Entrenamiento (Cómo abordar los ceros): Para calcular la probabilidad de una palabra dado un contexto, usas una de estas tres vías dependiendo de lo que exija el problema:

- **A. Suavizado de Laplace ($k = 1$):**

$$\mathbb{P}(w_m | \text{contexto}) = \frac{C(\text{contexto } w_m) + 1}{C(\text{contexto}) + (|V| + 1)}$$

(Suma 1 a la ocurrencia del bloque completo, y divide por el conteo del contexto puro más las continuaciones posibles).

- **B. Retroceso (Backoff):** Si la frecuencia del n-grama es 0, en lugar de dar probabilidad nula, el modelo asume por defecto la probabilidad del modelo inferior (ej. si falla el trigramma, usa el bigrama).
- **C. Interpolación:** Mezcla varios modelos asignando pesos λ que deben sumar 1:

$$\mathbb{P}(w_m | w_{m-1}) = \lambda_1 \mathbb{P}_{\text{unigrama}}(w_m) + \lambda_2 \mathbb{P}_{\text{bigrama}}(w_m | w_{m-1})$$

2. Predicción final de una frase completa: Para evaluar la probabilidad de que exista toda la frase generada, multiplicas las probabilidades condicionales acotadas o sumas sus logaritmos:

$$\mathbb{P}(w) \cong \prod_{m=1}^M \mathbb{P}(w_m | w_{m-(n-1)} \cdots w_{m-1})$$

O esta versión equivalente usando logaritmos para evitar underflow numérico:

$$\log \mathbb{P}(w) \cong \sum_{m=1}^M \log \mathbb{P}(w_m | w_{m-(n-1)} \dots w_{m-1})$$

CONFIGURACIÓN 4: Evaluación (Perplejidad)

Teniendo tu modelo de n-gramas anterior entrenado y un documento nuevo de prueba con N términos totales (incluyendo símbolos $\langle /s \rangle$), mides el desempeño del modelo usando base logarítmica:

$$\text{Perplejidad}(w) = 2^{-\frac{1}{N} \sum_{i=1}^M \log_2 \mathbb{P}(w_i)}$$

(Básicamente, tomas todas las probabilidades individuales que tu modelo calculó para la secuencia de prueba, les aplicas logaritmo en base 2, las sumas, divides por el número total de términos y elevas 2 a la menos todo eso).

Para resolver con éxito los ejercicios prácticos de esta sección, las fórmulas fundamentales se dividen en dos bloques principales: **Planificación bajo Incertidumbre** (donde el agente conoce la dinámica del entorno: las probabilidades de transición P y las recompensas R) y **Aprendizaje por Refuerzo** (donde el entorno es desconocido y el agente aprende interactuando con él por ensayo y error).

Aquí tienes el formulario maestro estructurado con las ecuaciones exactas y el significado de cada variable para aplicar en tu examen:

Bloque 1: Planificación bajo Incertidumbre (Procesos de Decisión de Markov)

Este bloque se resuelve aplicando métodos de programación dinámica cuando disponemos de un modelo completo del sistema.

1. Probabilidad de una historia (o trayectoria) parcial

Se utiliza para calcular la probabilidad de experimentar una secuencia específica de estados bajo una política de acciones π :

$$\mathbb{P}(h|\pi) = \prod_{i \geq 0} P_{\pi(s_i)}(s_{i+1}|s_i)$$

- s_i : Estado en el instante de tiempo i .
- $\pi(s_i)$: Acción que dicta la política aplicar en el estado s_i .
- $P_a(s'|s)$: Probabilidad de transitar al estado s' al aplicar la acción a en el estado s .

2. Evaluación de una política **Utilidad esperada** (Sistema de Ecuaciones Lineales de U_π)

Se aplica cuando te dan una política concreta π y te piden calcular la utilidad esperada de cada estado ($U_\pi(s)$). Genera un sistema de ecuaciones con tantas variables como estados tenga el problema:

$$U_\pi(s) = R(s, \pi(s)) + \gamma \sum_{s' \in S} P_{\pi(s)}(s'|s) U_\pi(s')$$

- $R(s, a)$: Recompensa neta del par estado-acción, calculada como $R(s) - C(s, a)$ (recompensa del estado menos el coste de la acción).
- γ : Factor de descuento ($0 < \gamma < 1$).
- $U_\pi(s')$: Utilidad esperada del posible estado futuro s' .

3. Ecuaciones de Bellman para la Utilidad Óptima (U^*)

Caracterizan el límite máximo teórico de utilidad que se puede conseguir en cada estado bajo la mejor política posible:

$$U^*(s) = \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s' \in S} P_a(s'|s) U^*(s') \right]$$

- *Nota de examen:* Es un sistema de ecuaciones no lineales debido al operador $\max$, por lo que se resuelve usando algoritmos iterativos.

4. Paso de actualización en Iteración de Valores (Value Iteration)

Fórmula recursiva para calcular la utilidad de la iteración $i + 1$ a partir de la obtenida en la iteración i :

$$U_{i+1}(s) = \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s' \in S} P_a(s'|s) U_i(s') \right]$$

- **Criterio de parada:** Se detiene cuando la diferencia máxima entre dos iteraciones consecutivas es menor que un margen de error ϵ :

$$\|U_i - U_{i-1}\| = \max_{s \in S} |U_i(s) - U_{i-1}(s)| < \epsilon$$

5. Cota Superior de la Utilidad (U_{max})

Define el límite asintótico máximo de utilidad acumulada si el agente recibiera siempre la recompensa máxima posible (R_{max}) en un horizonte infinito:

$$U_{max} = \frac{R_{max}}{1 - \gamma}$$

6. Extracción de la Política Óptima (π^*)

Una vez hallada la utilidad óptima (bien resolviendo Bellman o tras estabilizar la Iteración de Valores), la política óptima se extrae seleccionando la acción voraz:

$$\pi^*(s) = \arg \max_{a \in A(s)} \left[R(s, a) + \gamma \sum_{s' \in S} P_a(s'|s) U^*(s') \right]$$

Bloque 2: Aprendizaje por Refuerzo (Entorno Desconocido)

En estos ejercicios el agente no conoce las transiciones P ni las recompensas R , por lo que estima los valores directamente a partir de episodios (experiencia).

1. Utilidad del par Estado-Acción ($q_\pi(s, a)$)

Representa la utilidad esperada de aplicar la acción a en el estado s y, a partir de ahí, seguir la política π :

$$q_\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P_a(s'|s) U_\pi(s')$$

2. Actualización de Montecarlo para pares Estado-Acción

- **MC de primera visita:** Solo se toma la utilidad de la secuencia a partir de la primera aparición de un par (s, a) en el episodio.
- **MC de cada visita:** Se toman los valores de utilidad obtenidos a partir de todas las apariciones del par (s, a) en el episodio.
- **Fórmula incremental de la media (para actualización paso a paso):**

$$U^n(s) = U^{n-1}(s) + \frac{1}{n} (U_n - U^{n-1}(s))$$

Donde U_n es la utilidad real observada en el episodio actual para ese estado.

3. Método de las Diferencias Temporales (TD(0)) para Estados

Actualiza la estimación de la utilidad tras cada paso individual de interacción, sin necesidad de esperar a que termine todo el episodio:

$$U(s_t) \leftarrow U(s_t) + \alpha (R_t + \gamma U(s_{t+1}) - U(s_t))$$

- α : Factor de aprendizaje ($0 < \alpha \leq 1$).
- $R_t + \gamma U(s_{t+1}) - U(s_t)$: Es el denominado **Error DT** (δ_t), que mide la discrepancia entre la utilidad estimada del estado actual y la recompensa real obtenida más la utilidad estimada del estado siguiente.

4. Algoritmo Q-Learning (Diferencias Temporales para q)

Es el algoritmo de control off-policy más preguntado en los exámenes. Actualiza directamente la función de utilidad de pares estado-acción aproximando la óptima (q^*) independientemente de la política que se siga:

$$q(s, a) \leftarrow q(s, a) + \alpha \left[R + \gamma \max_{a' \in A(s')} q(s', a') - q(s, a) \right]$$

- s' : Estado siguiente observado tras aplicar a en s .
- $\max_{a'} q(s', a')$: El valor estimado de la mejor acción posible en el estado siguiente s' .